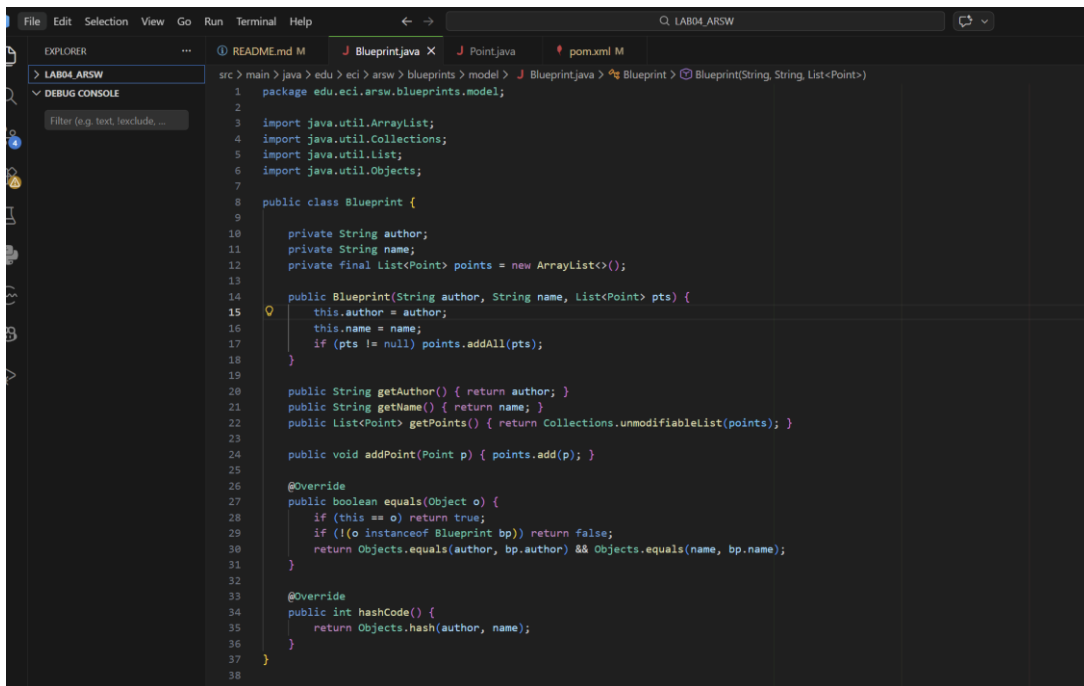


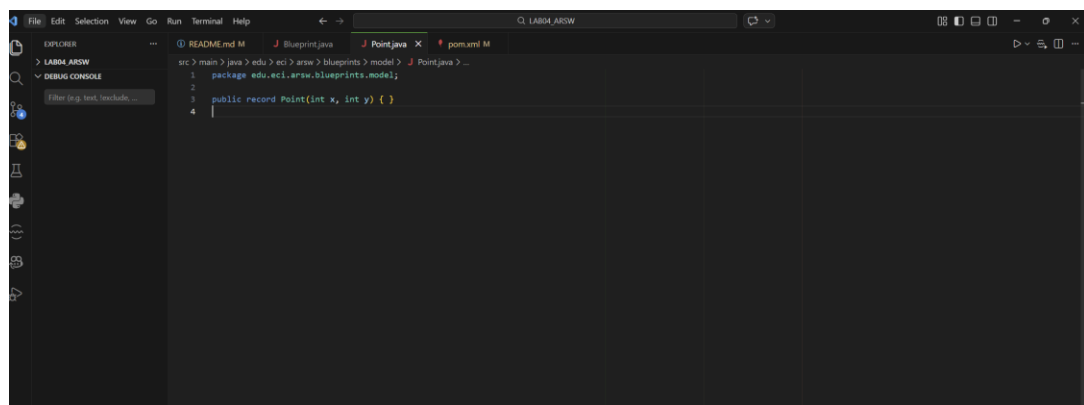
1. Familiarización con el código base

- Revisa el paquete model con las clases Blueprint y Point.



```
src > main > java > edu > eci > arsw > blueprints > model > Blueprint.java > Blueprint(String, String, List<Point>)
```

```
1 package edu.eci.arsw.blueprints.model;
2
3 import java.util.ArrayList;
4 import java.util.Collections;
5 import java.util.List;
6 import java.util.Objects;
7
8 public class Blueprint {
9
10     private String author;
11     private String name;
12     private final List<Point> points = new ArrayList<>();
13
14     public Blueprint(String author, String name, List<Point> pts) {
15         this.author = author;
16         this.name = name;
17         if (pts != null) points.addAll(pts);
18     }
19
20     public String getAuthor() { return author; }
21     public String getName() { return name; }
22     public List<Point> getPoints() { return Collections.unmodifiableList(points); }
23
24     public void addPoint(Point p) { points.add(p); }
25
26     @Override
27     public boolean equals(Object o) {
28         if (this == o) return true;
29         if (!(o instanceof Blueprint bp)) return false;
30         return Objects.equals(author, bp.author) && Objects.equals(name, bp.name);
31     }
32
33     @Override
34     public int hashCode() {
35         return Objects.hash(author, name);
36     }
37 }
38
```



```
src > main > java > edu > eci > arsw > blueprints > model > Point.java > ...
```

```
1 package edu.eci.arsw.blueprints.model;
2
3 public record Point(int x, int y) {}
4
```

Point

- Representa un punto en 2D con coordenadas x e y.
- Es inmutable (no se pueden cambiar sus valores después de crear el objeto).

Blueprint

- Representa un plano o diseño (blueprint) con un **autor**, un **nombre** y una **lista de puntos** (List<Point>).
- Permite agregar puntos uno a uno con el método addPoint(Point p).

- Entiende la capa persistence con InMemoryBlueprintPersistence.

```
@Repository
public class InMemoryBlueprintPersistence implements BlueprintPersistence {

    private final Map<String, Blueprint> blueprints = new ConcurrentHashMap<>();

    public InMemoryBlueprintPersistence() {
        // Sample data 1:1 style (author/name key)
        Blueprint bp1 = new Blueprint(author: "John", name: "house",
            List.of(new Point(x: 0,y: 0), new Point(x: 10,y: 0), new Point(x: 10,y: 10), new Point(x: 0,y: 10)));
        Blueprint bp2 = new Blueprint(author: "John", name: "garage",
            List.of(new Point(x: 5,y: 5), new Point(x: 15,y: 5), new Point(x: 15,y: 15)));
        Blueprint bp3 = new Blueprint(author: "Jane", name: "garden",
            List.of(new Point(x: 2,y: 2), new Point(x: 3,y: 4), new Point(x: 6,y: 7)));
        blueprints.put(keyOf(bp1), bp1);
        blueprints.put(keyOf(bp2), bp2);
        blueprints.put(keyOf(bp3), bp3);
    }

    private String keyOf(Blueprint bp) { return bp.getAuthor() + ":" + bp.getName(); }
    private String keyOf(String author, String name) { return author + ":" + name; }

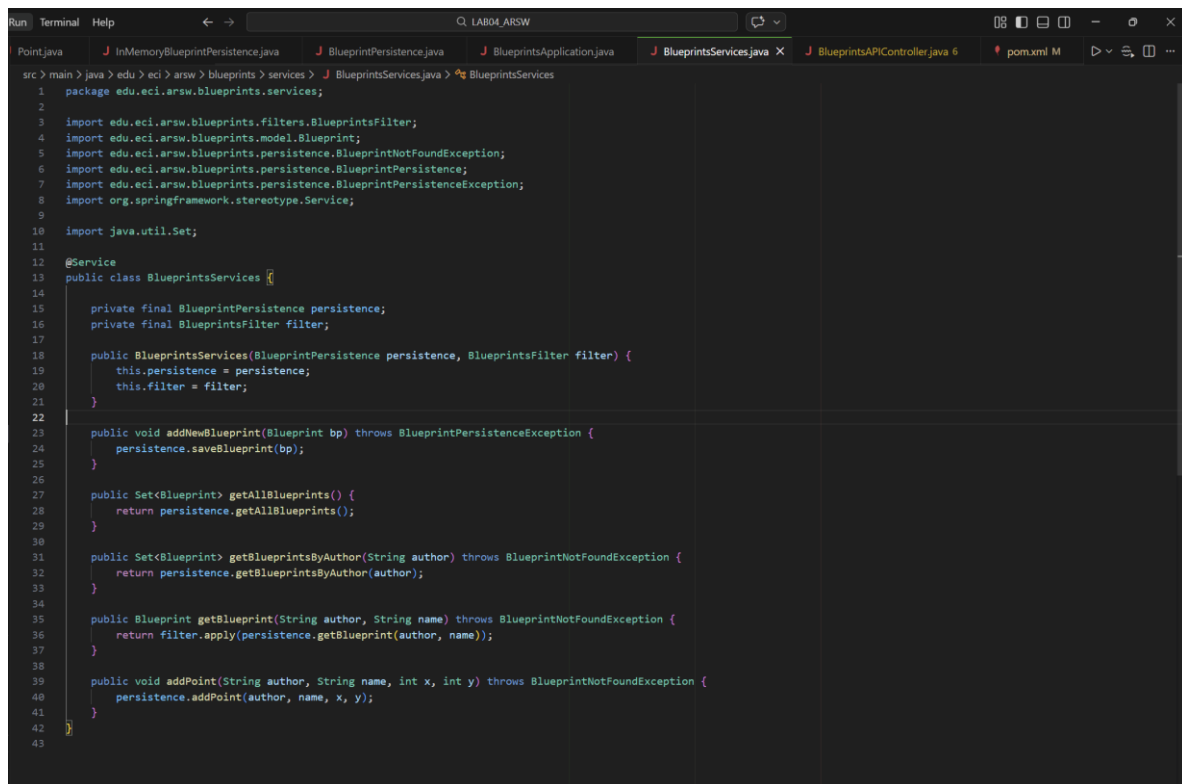
    @Override
    public void saveBlueprint(Blueprint bp) throws BlueprintPersistenceException {
        String k = keyOf(bp);
        if (blueprints.containsKey(k)) throw new BlueprintPersistenceException("Blueprint already exists: " + k);
        blueprints.put(k, bp);
    }

    @Override
    public Blueprint getBlueprint(String author, String name) throws BlueprintNotFoundException {
        Blueprint bp = blueprints.get(keyOf(author, name));
        if (bp == null) throw new BlueprintNotFoundException("Blueprint not found: %s/%s".formatted(author, name));
        return bp;
    }

    @Override
    public Set<Blueprint> getBlueprintsByAuthor(String author) throws BlueprintNotFoundException {
        Set<Blueprint> set = blueprints.values().stream()
            .filter(bp -> bp.getAuthor().equals(author))
            .collect(Collectors.toSet());
        if (set.isEmpty()) throw new BlueprintNotFoundException("No blueprints for author: " + author);
        return set;
    }
}
```

- Almacena los blueprints en memoria usando un mapa con clave autor:nombre.
- Permite crear, consultar, listar y actualizar blueprints y sus puntos.
- Lanza excepciones si se intenta acceder a un blueprint inexistente o duplicado.
- Incluye datos de ejemplo al inicializar para pruebas rápidas sin base de datos.

- Analiza la capa services (BlueprintsServices) y el controlador BlueprintsAPIController.



```
src > main > java > edu > eci > arsw > blueprints > services > J BlueprintsServices.java > BlueprintsServices
1 package edu.eci.arsw.blueprints.services;
2
3 import edu.eci.arsw.blueprints.filters.BlueprintsFilter;
4 import edu.eci.arsw.blueprints.model.Blueprint;
5 import edu.eci.arsw.blueprints.persistence.BlueprintNotFoundException;
6 import edu.eci.arsw.blueprints.persistence.BlueprintPersistence;
7 import edu.eci.arsw.blueprints.persistence.BlueprintPersistenceException;
8 import org.springframework.stereotype.Service;
9
10 import java.util.Set;
11
12 @Service
13 public class BlueprintsServices {
14
15     private final BlueprintPersistence persistence;
16     private final BlueprintsFilter filter;
17
18     public BlueprintsServices(BlueprintPersistence persistence, BlueprintsFilter filter) {
19         this.persistence = persistence;
20         this.filter = filter;
21     }
22
23     public void addNewBlueprint(Blueprint bp) throws BlueprintPersistenceException {
24         persistence.saveBlueprint(bp);
25     }
26
27     public Set<Blueprint> getAllBlueprints() {
28         return persistence.getAllBlueprints();
29     }
30
31     public Set<Blueprint> getBlueprintsByAuthor(String author) throws BlueprintNotFoundException {
32         return persistence.getBlueprintsByAuthor(author);
33     }
34
35     public Blueprint getBlueprint(String author, String name) throws BlueprintNotFoundException {
36         return filter.apply(persistence.getBlueprint(author, name));
37     }
38
39     public void addPoint(String author, String name, int x, int y) throws BlueprintNotFoundException {
40         persistence.addPoint(author, name, x, y);
41     }
42
43 }
```

Servicios (BlueprintsServices):

Expone métodos para interactuar con la persistencia y aplicar filtros (BlueprintsFilter).

Funciona como capa intermedia entre la persistencia y el controlador, manejando la lógica de negocio.

```
Run Terminal Help  ← →  Q LAB04_ARSW  [Icons] [Window] [File] [Edit] [Run] [Debug] [Test] [Tools] [Help]

Point.java x InMemoryBlueprintPersistence.java BlueprintPersistence.java BlueprintsApplication.java BlueprintsServices.java BlueprintsAPIController.java 6 x pom.xml M

src > main > java > edu > eci > arsw > blueprints > controllers > J BlueprintsAPIController.java > BlueprintsAPIController

1 package edu.eci.arsw.blueprints.controllers;
2
3 import edu.eci.arsw.blueprints.model.Blueprint;
4 import edu.eci.arsw.blueprints.model.Point;
5 import edu.eci.arsw.blueprints.persistence.BlueprintNotFoundException;
6 import edu.eci.arsw.blueprints.persistence.BlueprintPersistenceException;
7 import edu.eci.arsw.blueprints.services.BlueprintsServices;
8 import jakarta.validation.Valid;
9 import jakarta.validation.constraints.NotBlank;
10 import org.springframework.http.HttpStatus;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.web.bind.annotation.*;
13
14 import java.util.Map;
15 import java.util.Set;
16
17 @RestController
18 @RequestMapping("/blueprints")
19 public class BlueprintsAPIController {
20
21     private final BlueprintsServices services;
22
23     public BlueprintsAPIController(BlueprintsServices services) { this.services = services; }
24
25     // GET /blueprints
26     @GetMapping
27     public ResponseEntity<Set<Blueprint>> getAll() {
28         return ResponseEntity.ok(services.getAllBlueprints());
29     }
30
31     // GET /blueprints/{author}
32     @GetMapping("/{author}")
33     public ResponseEntity<> byAuthor(@PathVariable String author) {
34         try {
35             return ResponseEntity.ok(services.getBlueprintsByAuthor(author));
36         } catch (BlueprintNotFoundException e) {
37             return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", e.getMessage()));
38         }
39     }
40
41     // GET /blueprints/{author}/{bpname}
```

- Controlador (BlueprintsAPIController):

Expone endpoints REST. GET /blueprints devuelve todos los blueprints,

GET /blueprints/{author} devuelve los blueprints de un autor y

GET /blueprints/{author}/{bpname} devuelve un blueprint específico.

POST /blueprints permite crear un nuevo blueprint, mientras que PUT /blueprints/{author}/{bpname}/points agrega un punto a un blueprint existente.

El controlador maneja respuestas HTTP apropiadas (200 OK, 201 Created, 403 Forbidden, 404 Not Found)